

I. Processor Outline

```
input logic [7:0] opcode, // 8-bit opcode
input logic NFLG,        // Negative flag
input logic ZFLG,        // Zero flag
input logic reset,       // Reset signal
input logic clock,       // Clock signal
output logic [2:0] state, // 3-bit state output (State encoding)
output logic LOAC_AC,    // Load accumulator
output logic LOAC_IRU,   // Load upper byte of IR
output logic LOAD_IRL,   // Load lower byte of IR
output logic LOAD_PC,    // Load program counter
output logic INCR_PC,    // Increment program counter
output logic FETCH,      // Instruction fetch
output logic STORE_MEM   // Store data to memory
);

typedef enum logic [2:0] {
    START = 3'b000,
    PREP_U = 3'b001,
    FETCH_U = 3'b010,
    PREP_L = 3'b011,
    FETCH_L = 3'b100,
    EXECUTE = 3'b101
} state_t;

state_t current_state, next_state;

always_ff @(negedge clock or posedge reset) begin
    if (reset) begin
        current_state <= START;
    end else begin
        current_state <= next_state;
    end
end

always_ff @(current_state, opcode, NFLG, ZFLG) begin
    case (current_state)
        START: next_state = PREP_U;
        PREP_U: next_state = FETCH_U;
        FETCH_U: next_state = PREP_L;
        PREP_L: next_state = FETCH_L;
        FETCH_L: next_state = EXECUTE;
        EXECUTE: next_state = PREP_U;
        default: next_state = START;
    endcase
end
```

end

always_ff @(current_state) begin

```
    LOAC_AC = 0;  
    LOAC_IRU = 0;  
    LOAD_IRL = 0;  
    LOAD_PC = 0;  
    INCR_PC = 0;  
    FETCH = 0;  
    STORE_MEM = 0;
```

case (current_state)

```
    START: begin  
        LOAC_AC = 0;  
        LOAC_IRU = 0;  
        LOAD_IRL = 0;  
        LOAD_PC = 0;  
        INCR_PC = 0;  
        FETCH = 0;  
        STORE_MEM = 0;
```

end

PREP_U: begin

```
    LOAD_PC = 1; // Load PC  
    FETCH = 1;   // Fetch instruction
```

end

FETCH_U: begin

```
    LOAC_IRU = 1; // Load IRU with the fetched byte  
    INCR_PC = 1;  // Increment PC after fetching upper byte
```

end

PREP_L: begin

```
    LOAD_PC = 1; // Load PC again  
    FETCH = 1;   // Fetch instruction
```

end

FETCH_L: begin

```
    LOAD_IRL = 1; // Load IRL with the fetched byte  
    INCR_PC = 1;  // Increment PC after fetching lower byte
```

end

EXECUTE: begin

case (opcode)

8'h00: begin

```
    STORE_MEM = 0; // NOP (No operation)
```

end

8'h04: begin

```

        LOAC_AC = 1;  // CLR (Clear accumulator)
end

8'h02: begin
    LOAC_AC = 1;  // LOADI (Load immediate value)
end
8'h06: begin
    LOAC_AC = 1;  // ADDI (Add immediate)
end
8'h08: begin
    LOAC_AC = 1;  // SUBTI (Subtract immediate)
end
8'h14: begin
    LOAC_AC = 1;  // SHL (Shift left)
end
8'h15: begin
    LOAC_AC = 1;  // SHR (Shift right)
end

8'h01: begin
    LOAC_AC = 1;  // LOAD (Load from memory)
end
8'h05: begin
    LOAC_AC = 1;  // ADD (Add from memory)
end
8'h07: begin
    LOAC_AC = 1;  // SUBT (Subtract from memory)
end
8'h09: begin
    LOAC_AC = 1;  // NEG (Negate value in memory)
end
8'h0A: begin
    LOAC_AC = 1;  // NOT (Bitwise NOT)
end
8'h0B: begin
    LOAC_AC = 1;  // AND (Bitwise AND)
end
8'h0C: begin
    LOAC_AC = 1;  // OR (Bitwise OR)
end
8'h0D: begin
    LOAC_AC = 1;  // XOR (Bitwise XOR)
end

8'h0F: begin
    STORE_MEM = 1; // STORE (Store to memory)
end

```

```

        8'h10: begin
            LOAD_PC = 1; // JUMP (Jump to address)
        end
        8'h11: begin
            if (NFLG) LOAD_PC = 1; // JNEG (Jump if negative flag is
set)
        end
        8'h12: begin
            if (~NFLG | ZFLG) LOAD_PC = 1; // JPOSZ (Jump if
positive or zero flag is set)
        end
        8'h13: begin
            if (ZFLG) LOAD_PC = 1; // JZERO (Jump if zero flag is
set)
        end
        8'h14: begin
            if (~ZFLG) LOAD_PC = 1; // JNZER (Jump if not zero flag
is set)
        end

        default: begin
            STORE_MEM = 0; // Default: Do nothing or handle
invalid opcodes
        end
    endcase
end

    default: begin
        LOAC_AC = 0;
        LOAC_IRU = 0;
        LOAD_IRL = 0;
        LOAD_PC = 0;
        INCR_PC = 0;
        FETCH = 0;
        STORE_MEM = 0;
    end
endcase
end

    assign state = current_state;

endmodule

```

II. Processor Benchmarking

```

module control_tb(
    input logic [8:0] SW,
    input logic [0:0] KEY,
    output logic [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5,

```

```

    output logic [7:0] LEDs
);

    logic [7:0] opcode;
    logic [7:0] AC;
    logic [7:0] IRL;
    logic [7:0] IRU;
    logic [7:0] PC;
    logic [7:0] MAR;
    logic [7:0] MDR;

    logic [2:0] state;
    logic LOAC_AC, LOAC_IRU, LOAD_IRL, LOAD_PC, INCR_PC, FETCH,
STORE_MEM;

    control control1(
        .opcode(opcode),
        .NFLG(SW[0]),
        .ZFLG(SW[1]),
        .reset(SW[2]),
        .clock(KEY[0]),
        .state(state),
        .LOAC_AC(LOAC_AC),
        .LOAC_IRU(LOAC_IRU),
        .LOAD_IRL(LOAD_IRL),
        .LOAD_PC(LOAD_PC),
        .INCR_PC(INCR_PC),
        .FETCH(FETCH),
        .STORE_MEM(STORE_MEM)
    );

    dualseg7 dualdisplay0(
        .blank(0),
        .test(0),
        .datain(opcode),
        .disp0(HEX0),
        .disp1(HEX1)
    );

    dualseg7 dualdisplay1(
        .blank(0),
        .test(0),
        .datain(IRL),
        .disp0(HEX2),
        .disp1(HEX3)
    );

    dualseg7 dualdisplay2(

```

```

        .blank(0),
        .test(0),
        .datain(AC),
        .disp0(HEX4),
        .disp1(HEX5)
    );

    ledDriver ledDriver1(
        .values({STORE_MEM, FETCH, INCR_PC, LOAD_PC, LOAD_IRL,
        LOAD_IRU, LOAD_AC, state[2], state[1], state[0]}),
        .leds(LEDs)
    );

    always_ff @(posedge KEY[0] or posedge SW[2]) begin
        if (SW[2]) begin
            opcode <= 8'h00;
            AC <= 8'h00;
            IRL <= 8'h00;
            IRU <= 8'h00;
            PC <= 8'h00;
            MAR <= 8'h00;
            MDR <= 8'h00;
        end else if (SW[3] == 1) begin
            if (state == 3'b000) begin
                HEX0 <= opcode[7:4];
                HEX1 <= opcode[3:0];
                HEX2 <= IRL[7:4];
                HEX3 <= IRL[3:0];
                HEX4 <= AC[7:4];
                HEX5 <= AC[3:0];
                LEDs[9] <= 0;
            end else begin
                HEX0 <= PC[7:4];
                HEX1 <= PC[3:0];
                HEX2 <= MAR[7:4];
                HEX3 <= MAR[3:0];
                HEX4 <= MDR[7:4];
                HEX5 <= MDR[3:0];
                LEDs[9] <= 1;
            end
        end
    end

    always_ff @(state or LOAC_AC or LOAC_IRU or LOAD_IRL or LOAD_PC
    or INCR_PC or FETCH or STORE_MEM) begin
        LEDs[7:0] <= {STORE_MEM, FETCH, INCR_PC, LOAD_PC,
        LOAD_IRL, LOAD_IRU, LOAD_AC, state[2], state[1], state[0]};
    end

```

endmodule

